Gramáticas formales y lenguaje natural

Experimentando los límites de la jerarquía de Chomsky · Informe de Fase 1

Integrantes:

1. Rene Roberto Castañeda Mancia - 00360422
2. Diego Rafael Flores Serrato - 00029823
3. Luis Andrée Hernández Maltez - 00012321
4. Carlos Mauricio Canjura Paz - 00021622

Resumen

Se desarrolló un analizador descendente recursivo para un subconjunto de C y se comprobó su funcionamiento con programas válidos e inválidos. Después se construyeron tres gramáticas libres de contexto (CFG) para un fragmento controlado del español, incorporando progresivamente concordancia, modificadores y oraciones subordinadas. El objetivo fue distinguir entre errores corregibles ampliando la gramática, ambigüedades propias del análisis sintáctico y restricciones que, bajo una formalización particular, exceden el poder de una CFG. Los resultados se contrastaron con NLTK y spaCy sobre un conjunto común de oraciones.

1. Objetivo y enfoque

El propósito es identificar con precisión hasta dónde resulta adecuada una CFG y relacionar cada hallazgo con la jerarquía de Chomsky. Se siguieron cuatro pasos: (1) definir y ejecutar el parser de C; (2) extender la gramática al español mediante v1, v2 y v3; (3) documentar los puntos de falla L1–L6; y (4) comparar resultados con técnicas de procesamiento de lenguaje natural (NLP).

2. Marco teórico esencial

Una gramática formal describe reglas explícitas de formación; el lenguaje natural presenta variación, ambigüedad y concordancia entre elementos. Las CFG modelan estructuras recursivas y anidadas mediante reglas $A \rightarrow \alpha$. No obstante, expresar concordancias como género y número puede requerir múltiples variantes de una misma regla; y ciertas dependencias cruzadas, cuando se formalizan como lenguajes de copia, quedan fuera de las CFG.

Tipo	Clase / reconocedor	Relación con el proyecto
3 · Regular	Autómata finito	Análisis léxico: identificadores, números y operadores.
2 · Libre de contexto	Autómata de pila	Parser sintáctico, estructuras anidadas y CFG del español.
1 · Dependiente del contexto	Autómata linealmente acotado	Lenguajes de copia como {wcw}, estudiados para L6.
0 · Irrestricta	Máquina de Turing	Clase general que completa la jerarquía.

3. Parser formal: subconjunto de C

Se implementó en Python un parser descendente recursivo para programas con una función main, declaraciones de enteros, asignaciones, expresiones aritméticas, estructuras if/else, while, bloques y return. lexer.py reconoce tokens mediante expresiones regulares; parser.py aplica las producciones y core/arbol.py permite representar el árbol en texto, DOT o SVG. Se informan errores léxicos y sintácticos con línea y columna.

Gramática simplificada (EBNF)

```
programa   → 'int' 'main' '(' ')' bloque EOF
bloque     → '{' sentencia* '}'
sentencia  → declaracion | asignacion | retorno | si | mientras | bloque
declaracion → 'int' ID ('=' expresion)? ';'
asignacion → ID '=' expresion ';'
retorno    → 'return' expresion ';'
si         → 'if' '(' condicion ')' sentencia ('else' sentencia)?
mientras   → 'while' '(' condicion ')' sentencia
condicion  → expresion OPREL expresion
expresion  → termino (('+' | '-') termino)*
termino    → factor (('*' | '/') factor)*
factor     → NUM | ID | '(' expresion ')' | '-' factor
```

La separación expresión–término–factor impone precedencia aritmética. La implementación utiliza anticipación de un token para las alternativas principales, pero la gramática presentada no es estrictamente LL(1) por el conflicto del else colgante: el programa lo asocia al if más cercano.

Validación y errores

Categoría	Casos	Resultado esperado / observado
Programas válidos	12	Aceptados.
Inválidos también en C	14	Rechazados con diagnóstico.
Válidos en C, fuera del subconjunto	7	Rechazados intencionalmente por alcance.

Ejemplo: si falta el punto y coma tras int edad = 18, se obtiene «Se esperaba ‘;’; apareció ‘return’ en línea 3, columna 5». La tercera categoría evita confundir una restricción deliberada del lenguaje admitido con un error de funcionamiento.

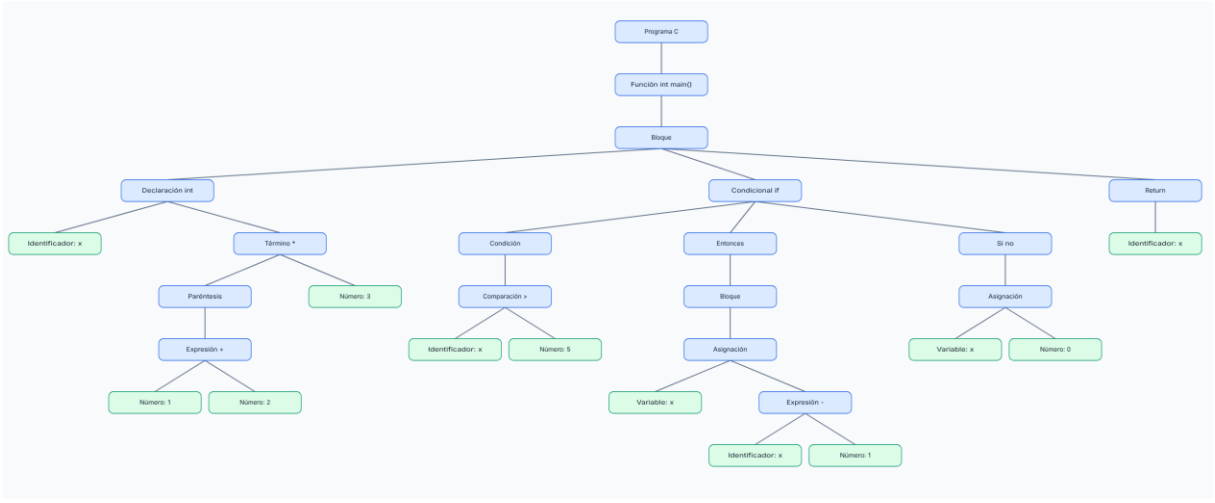


Figura 1. Árbol sintáctico generado por el parser de C: agrupación de expresiones y estructura condicional (imagen del proyecto original).

4. Extensión progresiva de la gramática al español

El análisis usa un léxico controlado de 95 formas flexionadas (sustantivos, adjetivos, determinantes, verbos, adverbios y preposiciones), principalmente en tercera persona. La gramática se amplió en tres iteraciones para observar qué construcciones se reconocen, qué errores se permiten y cuánto crece el número de reglas.

Versión	Producciones	Capacidad principal	Ejemplo de hallazgo
v1	5	Categorías sin concordancia	Acepta indebidamente «la gato negros duerme».
v2	22	Género y número en frase nominal y sujeto–verbo	Rechaza la discordancia y preserva la estructura básica.
v3	49	Adjetivos, adverbios, frases preposicionales y relativa	Reconoce «el gato que ve los ratones duerme»; conserva ambigüedad.

Producciones representativas de v3

```
O          → FN[g,n] FV[n]
FN[g,n]    → DET[g,n] NUC[g,n]
NUC[g,n]   → SUST[g,n] MODN[g,n]
MODN[g,n]  → ε | ADJ[g,n] MODN[g,n] | FPREP MODN[g,n]
           | SUBORD[n] MODN[g,n]
SUBORD[n]  → PRON_REL FV[n]
FV[n]      → NUCV[n] MODV
NUCV[n]    → V_INTR[n] | V_TRANS[n] FN[g',n']
MODV       → ε | ADV MODV | FPREP MODV
FPREP      → PREP FN[g,n]
```

Los rasgos [g,n] se codifican en los nombres de los no terminales. Así, una CFG finita puede imponer concordancia local duplicando reglas para cada combinación. El analizador del español utiliza retroceso y programación dinámica para contar derivaciones, ya que v3 permite más de un árbol para ciertas oraciones.

Pruebas del fragmento

El conjunto de validación contiene 51 oraciones en tres niveles: 20 de estructura y concordancia básica, 18 con modificadores y 13 con relativas o fenómenos complejos. Los resultados coincidieron con las etiquetas esperadas del diseño en los 51 casos; tres oraciones gramaticales se rechazan deliberadamente por estar fuera del alcance de v3.

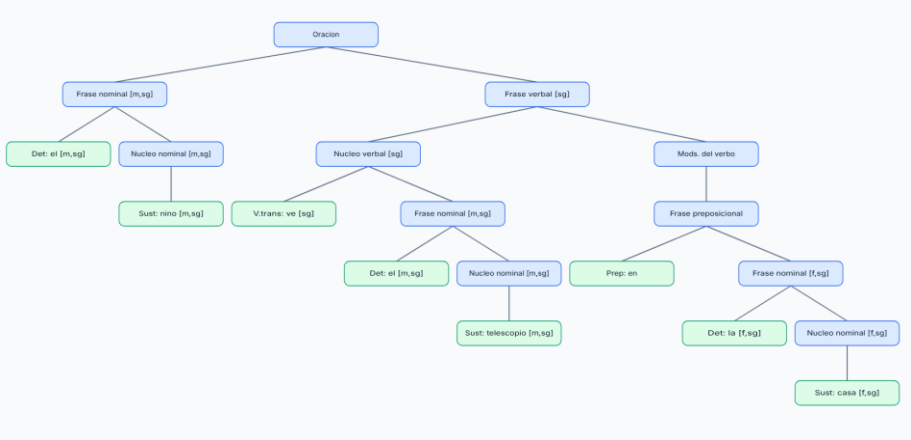


Figura 2. Una interpretación de «el niño ve el telescopio en la casa»: la frase preposicional modifica al verbo; existe otra derivación en la que modifica al sustantivo.

5. Puntos de falla y relación con Chomsky

Se identificaron seis fenómenos. Es fundamental diferenciar sobregeneración, ambigüedad y aumento de tamaño de la gramática de una limitación demostrada de poder generativo. En cinco casos, el comportamiento puede representarse con una CFG; L6 analiza una formalización más exigente.

ID	Fenómeno / regla implicada	Hallazgo	Clase
L1	Concordancia ausente en FN → DET SUST ADJ (v1)	Sobregeneración: acepta rasgos incompatibles.	Tipo 2
L2	Replicación de reglas nominales con rasgos (v2/v3)	Coste de codificar género y número; no imposibilidad.	Tipo 2
L3	MODN → FPREP MODN / MODV → FPREP MODV	Ambigüedad por adjunción de frase preposicional.	Tipo 2
L4	«baja» puede ser adjetivo o verbo	Ambigüedad léxica controlada.	Tipo 2
L5	MODN → SUBORD MODN	Concordancia separada por subordinada anidada.	Tipo 2
L6	Coordinación distributiva con «respectivamente»	Dependencias cruzadas modeladas como lenguaje copia.	Tipo 1*

**** L6 se clasifica así bajo la formalización de lenguaje copia adoptada en el experimento; no es una prueba automática sobre todo el español natural.***

Concordancia: coste finito (L1–L2)

La concordancia de género × número genera cuatro combinaciones básicas. Las producciones aumentan de 5 (v1) a 22 (v2) y 49 (v3). Al añadir persona como rasgo de simulación, la versión v3 alcanzaría 189 producciones. Es una expansión considerable, pero sigue siendo una CFG finita. No debe confundirse complejidad de ingeniería con un cambio de clase en Chomsky.

Ambigüedad y distancia (L3–L5)

Al añadir 0, 1, 2 y 3 frases preposicionales a la familia estudiada se obtuvieron 1, 2, 5 y 14 árboles sintácticos, respectivamente (serie de Catalan). Una CFG puede generar las distintas lecturas, aunque no elegir la apropiada sin conocimiento adicional. También puede mantener la concordancia en una estructura anidada como «el gato que ve los ratones duerme»: el rasgo singular del sujeto se conserva a través de la subordinada.

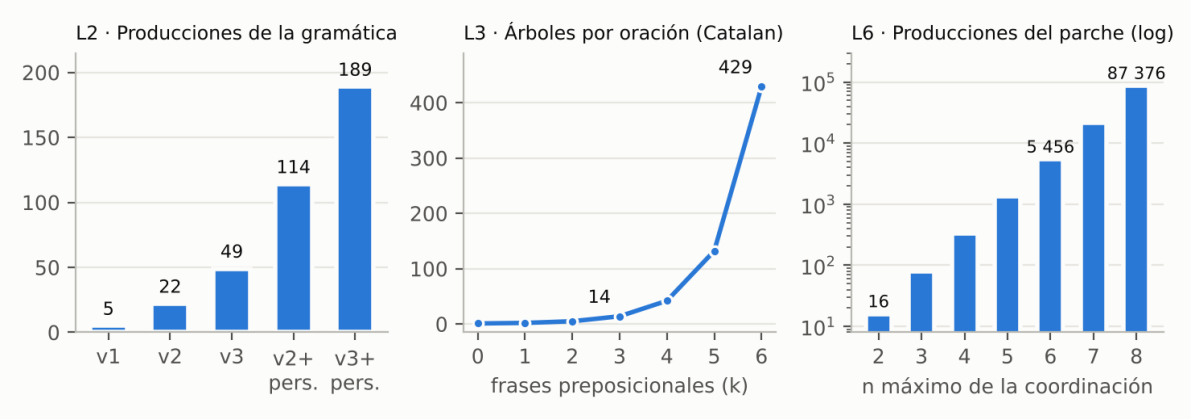


Figura 3. Evolución registrada en el proyecto: tamaño de gramática, número de árboles y coste de un parche acotado para L6.

6. Análisis formal del caso L6

La construcción estudiada es «el niño y la niña son alto y baja respectivamente», donde cada adjetivo debe corresponder al sustantivo de la misma posición. En una secuencia ampliable de sujetos y adjetivos, las correspondencias pueden representarse como dependencias cruzadas. El proyecto implementó un parche para coordinaciones acotadas; al aumentar el número de pares, debe añadir explícitamente nuevas combinaciones.

Reducción a un lenguaje de copia

Bajo el modelo formal del experimento, cada combinación de género y número se representa por un símbolo de un alfabeto finito Σ. La secuencia de rasgos de los sujetos y la de los adjetivos deben coincidir en orden. Tras eliminar los tokens fijos y preservar los rasgos correspondientes, se obtiene el lenguaje abstracto:

$$L = \{ w\ c\ w \mid w \in \Sigma^*, |w| \geq 2 \}$$

Este lenguaje no es libre de contexto. La idea de la demostración consiste en suponer que es una CFG, elegir una cadena con dos copias idénticas y aplicar el lema de bombeo para obtener una contradicción. Se considera $z = a^p b^p c a^p b^p$, donde p es la longitud de bombeo.

Esquema del argumento por casos

Ubicación de la parte bombeada	Consecuencia de bombear
Incluye el separador c	Puede eliminarse o repetirse el único separador.
Queda en la primera copia	Solo cambia la primera secuencia de símbolos.
Queda en la segunda copia	Solo cambia la segunda secuencia.
Cruza el separador sin incluirlo en los segmentos repetidos	Modifica el final de la primera copia y el inicio de la segunda, sin preservar la igualdad.

Por tanto, toda descomposición permitida puede bombearse de modo que la cadena resultante deje de pertenecer a L. El lenguaje de copia sirve como ejemplo de una restricción de tipo 1 que una CFG general no reconoce. Para cualquier longitud máxima fija, en cambio, sí puede construirse una CFG que enumere los casos admitidos.

Alcance y cautela lingüística

La demostración establece la propiedad del lenguaje abstracto utilizado. Para trasladarla al español natural sería necesario justificar independientemente que todas las correspondencias impuestas por «respectivamente» son restricciones gramaticales obligatorias y que la reducción conserva exactamente las oraciones pertinentes. La literatura lingüística ha discutido este tipo de argumentos (Pullum y Gazdar, 1982). Por ello, el proyecto presenta L6 como un límite del fragmento formalizado y no como una demostración concluyente sobre el español completo.

Conclusión formal de la sección

El experimento separa dos fronteras: (a) concordancia y subordinación anidada, que siguen dentro de las CFG aunque encarezcan el diseño, y (b) igualdades de secuencias arbitrariamente largas, que en la formalización seleccionada requieren mayor poder expresivo.

7. Pruebas y comparación con herramientas NLP

El conjunto comparativo contiene 61 oraciones: 51 del equipo y 10 adicionales de español fuera del vocabulario o de las construcciones de v3. Se emplearon dos etiquetas por oración: «salida esperada para v3» y «gramaticalidad en español». La comparación cuantitativa utiliza esta segunda etiqueta; de otro modo, los rechazos deliberados ocultarían falsos negativos.

Resultados sobre 61 oraciones

Sistema	TP	FP	FN	TN	Precisión	Cobertura	Rechazo cor.	Exactitud
v1	17	11	28	5	0.607	0.378	0.312	0.361
v2	17	0	28	16	1.000	0.378	1.000	0.541
v3	32	0	13	16	1.000	0.711	1.000	0.787
spaCy + regla	44	7	1	9	0.863	0.978	0.562	0.869

TP: verdadero positivo; FP: falso positivo; FN: falso negativo; TN: verdadero negativo. Métricas transcritas del documento de resultados del equipo.

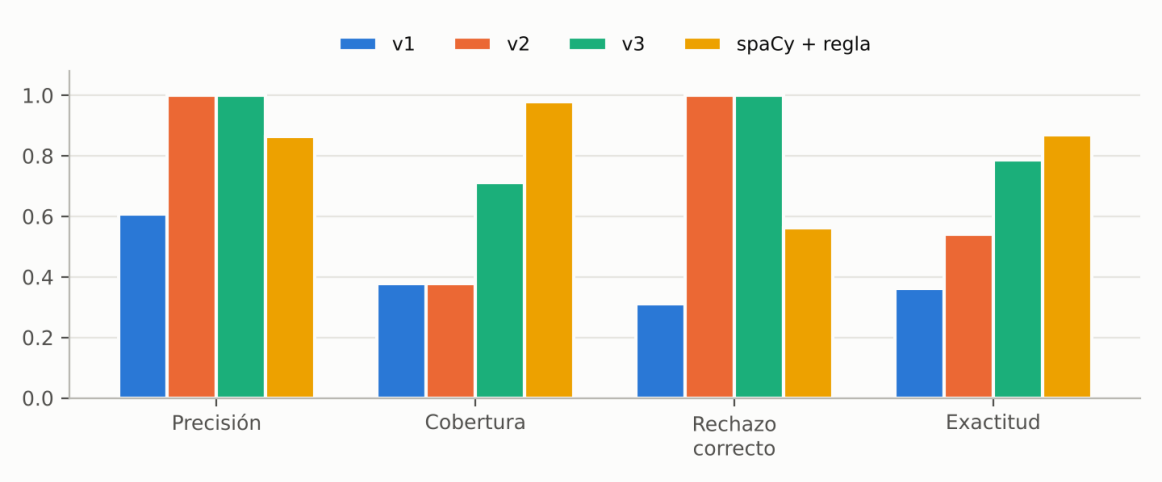


Figura 4. Comparación gráfica de precisión, cobertura, rechazo correcto y exactitud (fuente: experimentos del equipo).

Interpretación

De v1 a v2 desaparecen 11 falsos positivos, lo que muestra la utilidad de controlar la concordancia. Con v3 se aceptan 15 oraciones gramaticales adicionales frente a v2, sin aumentar los falsos positivos del conjunto estudiado. spaCy con una regla de concordancia alcanza una mayor cobertura (0.978), mientras que v3 presenta un rechazo correcto superior (1.000 frente a 0.562). Es una comparación entre tareas distintas: spaCy genera análisis incluso para entradas agramaticales y el rechazo fue añadido mediante una regla construida por el equipo.

Verificación independiente con NLTK

La gramática v3 se exportó a nltk.EarleyChartParser. En 61/61 oraciones se registró el mismo número de derivaciones que en el parser propio, incluida la serie de ambigüedad 1–2–5–14. NLTK funciona aquí como verificación cruzada de la CFG, no como un segundo modelo estadístico de español.

8. Discusión de resultados y limitaciones

Los 13 falsos negativos de v3 se atribuyeron a: siete oraciones fuera del vocabulario, tres construcciones no cubiertas, dos coordinaciones generales y un caso modelado como dependencia cruzada. Por tanto, la mayor parte de los rechazos de oraciones gramaticales se explica por el alcance limitado del prototipo, no por la imposibilidad de toda CFG.

En spaCy, siete oraciones agramaticales fueron aceptadas tras aplicar la regla de concordancia. El análisis del equipo encontró rasgos morfológicos ausentes, rasgos predichos a partir del contexto y dificultades para seguir la relación entre una subordinada y su antecedente. Los resultados dependen del corpus pequeño, del vocabulario seleccionado, de la propia regla complementaria y del modelo utilizado: no permiten una conclusión universal de superioridad entre enfoques.

9. Reproducibilidad del proyecto

El repositorio reúne el parser de C, las extensiones al español, 33 programas de validación, 61 oraciones comparativas, árboles exportables y pruebas automatizadas. En el documento del equipo se reportan 89 pruebas unitarias correctas. La ejecución se realiza desde la raíz del repositorio con Python 3.10 o posterior; la comparación completa requiere instalar NLTK, spaCy y el modelo es_core_news_sm.

```
python -m fase1_parser_formal.cli --archivo validacion/programa_valido.c
python -m fase1_parser_formal.cli --archivo validacion/programa_invalido.c
python -m fase1_extension_espanol.cli --oracion "el gato que ve los ratones duerme"
python -m comparacion_nlp.ejecutar_pruebas_c
python -m comparacion_nlp.comparar
```

Repositorio: https://github.com/Rene545/Proyecto_TEO · Las instrucciones completas y las dependencias están descritas en README.md.

10. Conclusiones

- El parser descendente recursivo reconoce el subconjunto de C especificado, genera árboles sintácticos y señala errores con posición; los 33 casos del corpus obtuvieron el comportamiento previsto.
- La concordancia básica y la subordinación anidada pueden modelarse mediante CFG; el coste práctico aumenta cuando los rasgos deben codificarse como variantes de reglas.
- La ambigüedad estructural puede multiplicar los árboles de análisis sin que la oración deje de pertenecer a una CFG. Resolver la lectura correcta exige información adicional.
- El lenguaje de copia usado para estudiar L6 permite exhibir una frontera formal, pero la extrapolación al español natural requiere supuestos lingüísticos adicionales.
- La comparación con NLP muestra una compensación entre cobertura y rechazo correcto. El análisis debe basarse en el tipo de herramienta, el conjunto de prueba y el alcance de la gramática.